

LAPORAN TUGAS BESAR
PEMROGRAMAN BERORIENTASI OBJEK

BATTLE OF HEROES



KELOMPOK 2:

Doni Agus Setiawan	123140009
Aprililianti	123140041
Nabila Ramadhani Mujahidin	123140062
Yuni Okta Safitri	123140213

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
2025

DAFTAR ISI

DAFTAR ISI	2
A. DESKRIPSI PENUGASAN	3
B. NAMA GAME.....	3
C. KATEGORI GAME	3
D. ENTITAS.....	4
E. DESKRIPSI GAME	5
F. TAMPILAN GAME.....	8
G. KELAS	10
H. OBJEK.....	11
I. ENKAPSULASI.....	13
J. PEWARISAN.....	13
K. POLIMORFISME.....	14
L. ABSTRAKSI.....	14
M. PENANGANAN KESALAHAN	15
N. GRAPHICAL USER INTERFACE	15
O. DIAGRAM UNIFIED MODELING LANGUAGE.....	16
P. KODE.....	19
Q. LAMPIRAN	26

A. DESKRIPSI PENUGASAN

Nama	NIM	Tugas
Doni Agus Setiawan	123140009	1. Kode/program 2. Presentasi 3. Sound game
Aprililianti	123140041	1. Presentasi 2. Laporan/Slide 3. UML
Nabila Ramadhani Mujahidin	123140062	1. Desain Program, 2. Presentasi 3. Kode/Program
Yuni Okta Safitri	123140213	1. Desain Program. 2. Presentasi 3. Laporan/Slide.

B. NAMA GAME

"Battle of Heroes" adalah game aksi 2D bertema fantasi yang menyajikan pengalaman bermain sederhana namun seru. Meskipun tampilannya ringan, game ini menawarkan sistem pertarungan real-time yang dinamis, bukan berbasis giliran (bukan turn-based). Pemain mengendalikan seorang pahlawan yang harus mengalahkan monster dengan menggunakan berbagai jenis serangan sihir seperti Fireball, Ice, dan Lightning.

C. KATEGORI GAME

Game Battle of Heroes termasuk dalam kategori action shooter side-scroller. Permainan ini menggabungkan elemen aksi cepat (action) dan mekanisme penembakan (shooter), di mana pemain mengendalikan seorang pahlawan yang bergerak secara vertikal dalam satu layar, dan menembakkan berbagai jenis proyektil seperti Fireball, Ice, dan Lightning untuk mengalahkan musuh yang datang dari sisi kanan layar.

Meskipun tidak bergerak scrolling penuh seperti platformer klasik, game ini tetap dikategorikan sebagai side-scroller statis karena arah datangnya musuh dan tembakan semuanya bergerak secara horizontal.

Kami memilih kategori ini karena gameplay-nya menekankan pada:

- Refleks pemain dalam menghindari musuh,
- Strategi penyerangan menggunakan jenis serangan yang berbeda,
- Manajemen cooldown dan posisi, serta
- Ketahanan karakter, dengan sistem HP dan sistem level dinamis.

Adanya sistem skor, peningkatan kecepatan proyektil, dan penurunan cooldown seiring naiknya level juga memperkuat unsur tantangan dan meningkatkan intensitas aksi selama permainan berlangsung.

D. ENTITAS

Entitas	Deskripsi
Pahlawan (Player)	Karakter utama yang dikendalikan pemain. Dapat bergerak vertikal, menembakkan proyektil (Fireball, Ice, Lightning), dan bertahan dari serangan musuh. HP akan berkurang jika terkena musuh.
Musuh (Enemy)	Karakter musuh yang muncul dari sisi kanan layar dan bergerak ke arah pemain. Jika menyentuh pemain atau keluar dari layar, akan mengurangi HP pemain.
Tempat (Arena)	Latar pertempuran berupa gambar statis (background) tempat aksi berlangsung. Tidak memiliki pengaruh langsung pada gameplay, namun memperkuat atmosfer.
Proyektil (Fireball, Ice, Lightning)	Objek serangan yang ditembakkan oleh pemain. Memiliki gambar, kecepatan, efek visual berbeda, dan cooldown masing-masing.
Game Engine	Logika utama permainan, terdiri dari game loop, mekanisme update objek (musuh, proyektil), rendering, dan penanganan input (keyboard).

User (Pemain)	Manusia yang mengendalikan permainan dengan menekan tombol panah (↑/↓) dan tombol aksi (Z, X, C) untuk menembak.
Sistem (Pygame)	Framework yang digunakan untuk menjalankan keseluruhan game, termasuk rendering grafis, pengelolaan event, dan audio.
Skor (Score)	Nilai yang menunjukkan jumlah musuh yang berhasil dikalahkan. Skor naik setiap kali musuh terkena proyektil. Peningkatan level terjadi setiap 10 poin.
Menu Utama (Main Menu)	Tampilan awal permainan dengan opsi navigasi seperti Start, Info, dan Exit. Dikendalikan oleh input keyboard.
Latar Musik (Background)	Audio dalam game ini mencakup background untuk membangun suasana, efek tembakan air, api, dan cahaya yang disesuaikan dengan jenis serangannya, serta sound game over bernuansa kegagalan. Semua elemen suara dirancang untuk mendukung visual dan meningkatkan pengalaman bermain.

E. DESKRIPSI GAME

Alur Game

1. Menu Utama

- Saat game dijalankan, pemain masuk ke layar menu utama yang menampilkan tiga opsi:
 - Start: Memulai permainan.
 - Info: Menampilkan informasi tim dan kontrol permainan.
 - Exit: Keluar dari game.
- Pemain dapat menavigasi menu menggunakan tombol ↑ / ↓ dan memilih opsi dengan Enter.

2. Gameplay Dimulai

- Saat memilih Start, permainan dimulai dari level 1.
- Pemain mengontrol karakter utama (penyihir) dan mulai menghadapi musuh yang datang dari sisi kanan layar.

3. Pertarungan

- Musuh (goblin) akan muncul setiap detik (`spawn_enemy()`), bergerak ke kiri.
- Pemain dapat menembak:
 - Z untuk Fireball (tanpa cooldown).
 - X untuk Ice (dengan cooldown).
 - C untuk Lightning (dengan cooldown).
- Proyektil yang mengenai musuh akan menghancurkan musuh dan menambah skor.
- Jika musuh menyentuh pemain atau keluar dari sisi kiri layar, pemain kehilangan HP.

4. Level dan Kesulitan

- Skor yang dikumpulkan memengaruhi level permainan:
 - Naik level setiap 10 poin.
 - Kecepatan fireball bertambah.
 - Cooldown Ice dan Lightning dikurangi, membuat permainan lebih cepat dan menantang.

5. Game Over

- Jika HP pemain mencapai 0, maka layar Game Over ditampilkan.
- Pemain dapat:
 - Menekan Enter untuk kembali ke menu utama.
 - Menekan ESC untuk keluar dari game.

Cara Kerja dan Perilaku Game

Komponen	Penjelasan
Player	Dapat bergerak vertikal dan menembakkan tiga jenis sihir dengan tombol Z, X, C.
Projectile	Menggunakan konsep pewarisan OOP. Semua sihir adalah subclass dari Projectile.
Musuh(Enemy)	Spesifikasi berupa <code>pygame.Rect</code> . Bergerak ke kiri, memberi damage jika menyentuh

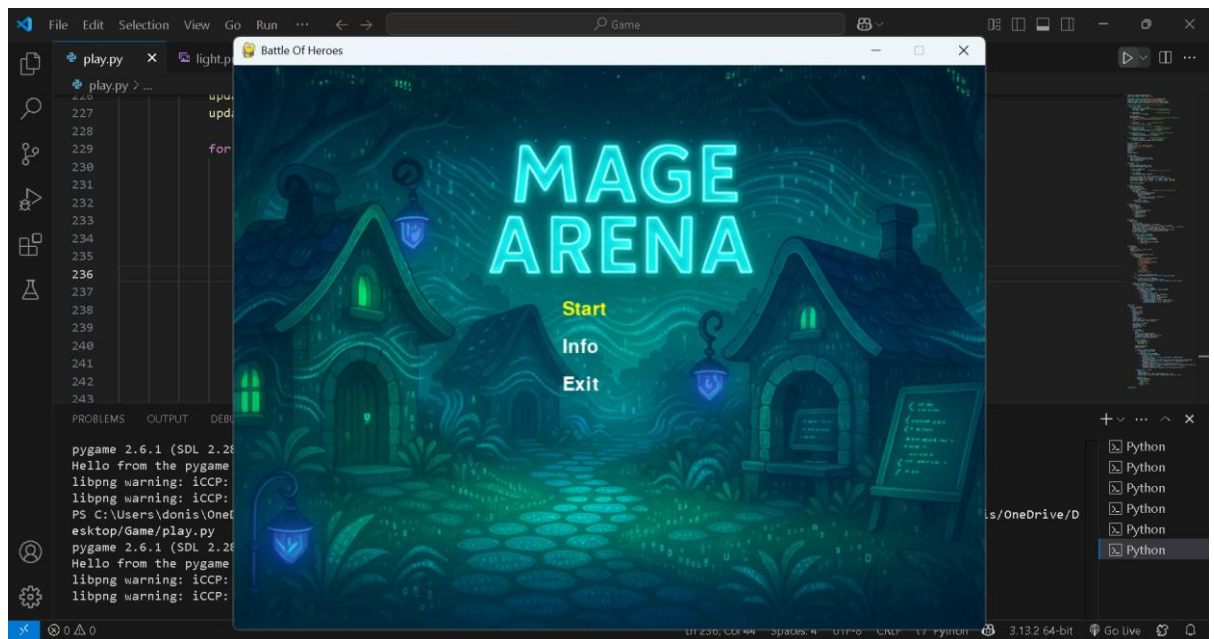
	pemain atau lolos.
Score & Level	Setiap musuh yang dikalahkan menambah skor. Level naik setiap 10 skor.
Cooldown	Ice dan Lightning hanya bisa digunakan setelah cooldown-nya habis.
HP bar	Ditampilkan di layar. Berkurang saat musuh menyentuh pemain atau lolos.
Game Over	Muncul jika $HP \leq 0$. Tampilkan skor akhir dan opsi untuk kembali ke menu atau keluar.

Kondisi-Kondisi dalam Game

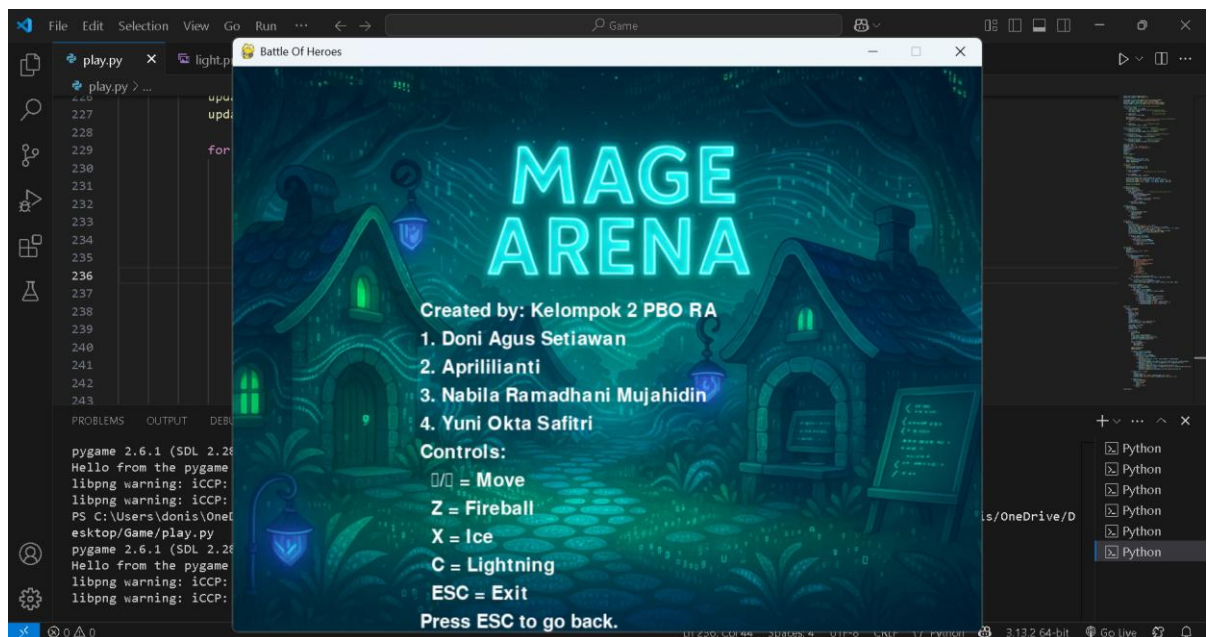
Kondisi	Penjelasan
<code>player_hp <= 0</code>	Game Over.
<code>score // 10 + 1 > level</code>	Naik level, tingkat kesulitan meningkat.
<code>now - cooldowns["ice"] > cooldown_time["ice"]</code>	Ice bisa ditembakkan kembali.
<code>proj.rect.colliderect(enemy)</code>	Musuh terkena serangan dan dihancurkan.
<code>proj.rect.x > WIDTH</code>	Proyektil keluar dari layar, dihapus dari list.
<code>enemy.colliderect(player_rect)</code>	Musuh menyentuh pemain, HP berkurang 10.
<code>enemy.x < 0</code>	Musuh lolos, HP berkurang 5.

Game ini menggabungkan konsep arcade shooter dengan elemen RPG ringan seperti sihir dan level up. Dengan menggabungkan sistem skor, cooldown, dan progresi level, game Battle of Heroes memberikan tantangan yang bertahap sambil melatih refleks dan strategi pemain dalam menggunakan berbagai jenis serangan sihir.

F. TAMPILAN GAME



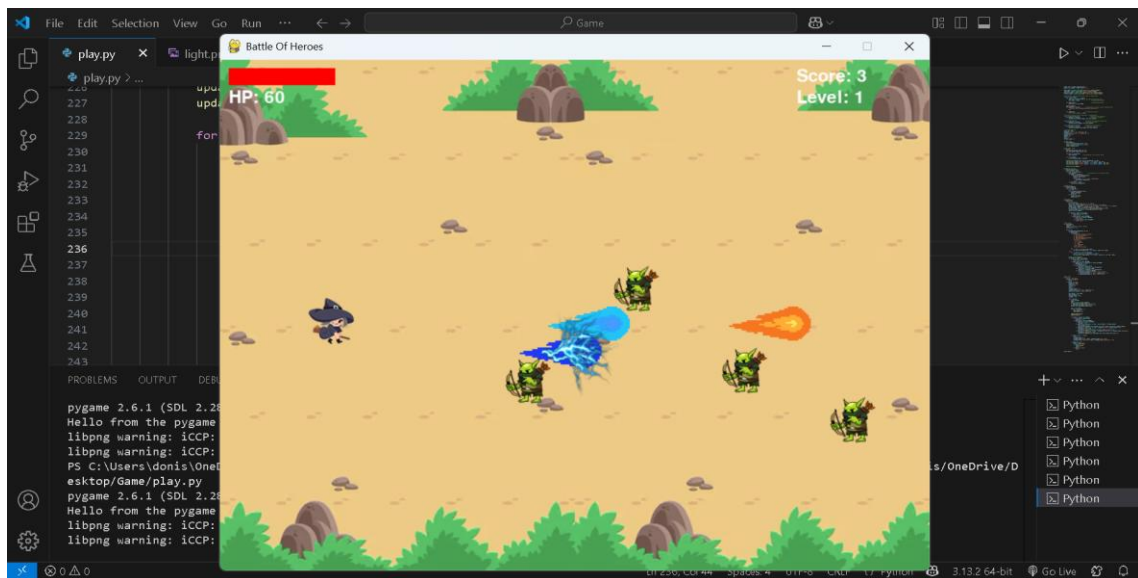
Gambar 1.Tampilan Menu



Gambar 2.Tampilan Info Game



Gambar 3.Tampilan Background Arena



Gambar 4.Tampilan Dalam Game

G. KELAS

1. Projectile
 - **Tipe:** Kelas abstrak (abstract class)
 - **Peran:** Menjadi kelas dasar bagi semua jenis proyektil (sihir yang ditembakkan)
 - **Ciri:**
 - Menyimpan data posisi dan kecepatan proyektil.
 - Memiliki metode update() untuk menggerakkan proyektil ke kanan.
 - Memiliki metode abstrak draw() yang wajib diimplementasikan oleh setiap subclass untuk menampilkan gambar proyektil yang sesuai.
 - Mendukung **polimorfisme** melalui metode type() yang mengembalikan nama kelas proyektil.
2. Fireball
 - **Tipe:** Kelas turunan dari Projectile
 - **Peran:** Proyektil jenis api.
 - **Ciri:**
 - Menampilkan gambar fireball saat ditembakkan.
 - Tidak memiliki cooldown (dapat ditembakkan dengan cepat).
 - Kecepatan dapat meningkat seiring naiknya level.
3. Ice
 - **Tipe:** Kelas turunan dari Projectile
 - **Peran:** Proyektil jenis es.
 - **Ciri:**
 - Menampilkan gambar es.
 - Memiliki efek serangan khusus (tidak langsung dihancurkan setelah mengenai musuh).
 - Memiliki cooldown sedang (default 3 detik, berkurang saat naik level).
4. Lightning
 - **Tipe:** Kelas turunan dari Projectile
 - **Peran:** Proyektil jenis petir.
 - **Ciri:**
 - Menampilkan gambar kilat saat ditembakkan.
 - Memiliki cooldown paling lama (default 5 detik, berkurang saat naik level).
 - Digunakan untuk serangan kuat dan strategis.

H. OBJEK

Hero (2D)

- Deskripsi: Karakter utama yang dikendalikan oleh pemain.
- Peran:
 - Bergerak ke atas dan bawah di layar.
 - Menyerang musuh menggunakan tiga jenis proyektil: Fireball, Ice, dan Lightning.
 - Memiliki HP (nyawa) yang berkurang jika terkena musuh.
 - Naik level secara otomatis setiap 10 skor, yang akan meningkatkan kecepatan serangan dan mengurangi waktu cooldown.

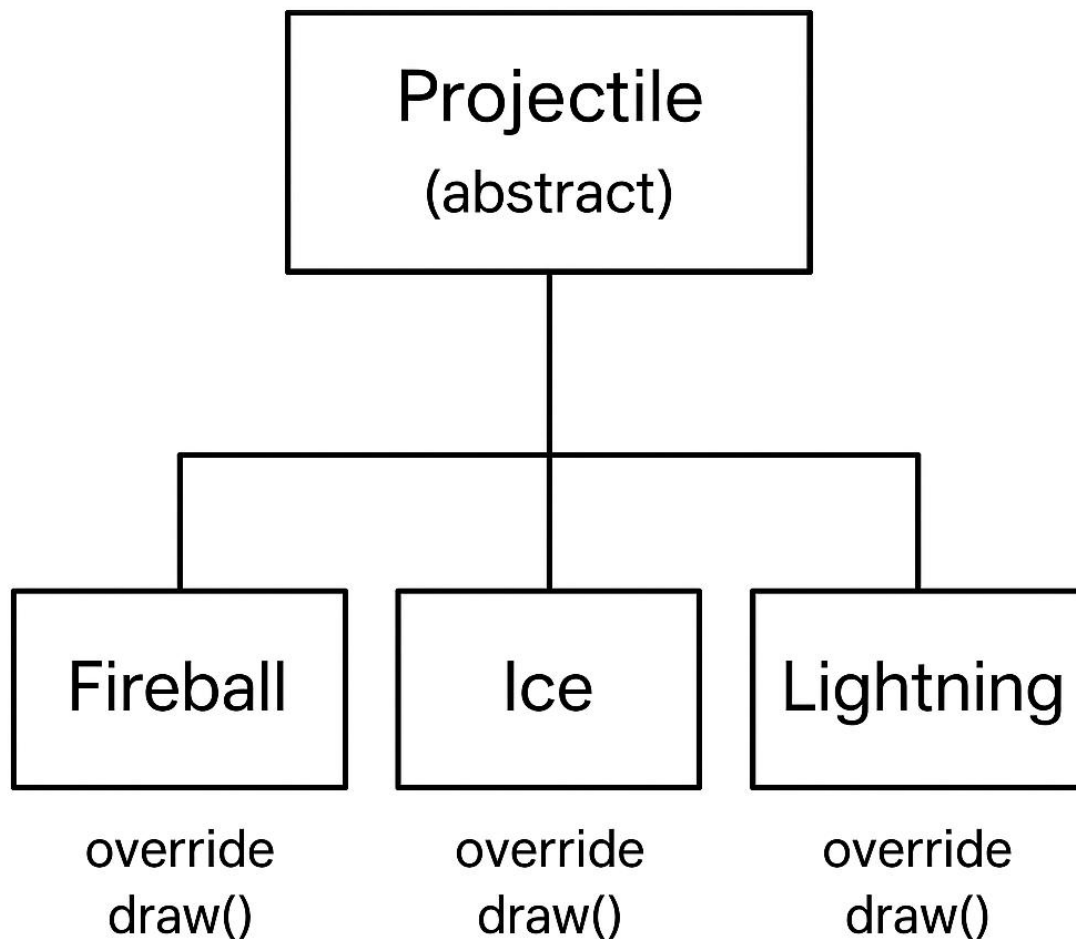
Monster (2D)

- Deskripsi: Musuh yang muncul secara berkala dari sisi kanan layar.
- Peran:
 - Bergerak menuju Hero untuk menyerang.
 - Jika menyentuh Hero atau melewati batas kiri layar, akan mengurangi HP Hero.
 - Bisa dihancurkan dengan proyektil yang ditembakkan oleh Hero.

Arena (2D)

- Deskripsi: Latar belakang atau tempat berlangsungnya pertempuran.
- Peran:
 - Sebagai medan tempur tempat Hero dan Monster bertarung.
 - Ditampilkan sebagai gambar latar belakang untuk memperkuat suasana pertempuran.

Objek Diagram



Konsep Object-Oriented Programming (OOP), khususnya untuk aspek berikut:

1. Kelas dan Pewarisan:

- Kelas abstrak Projectile dijadikan dasar (superclass).
- Kelas turunan Fireball, Ice, dan Lightning mewarisi dari Projectile.

2. Polimorfisme:

- Metode draw() diimplementasikan ulang (di-override) di masing-masing kelas turunan. Ketika metode draw() dipanggil, implementasi spesifik dari kelas yang dipakai akan digunakan, tanpa harus mengetahui jenis peluru.

3. Encapsulation (Enkapsulasi):

- Atribut dan metode (seperti posisi proyektil dan logika pergerakannya) dikemas dalam satu objek, sehingga detail internal tidak terlihat langsung oleh pengguna kelas.

4. Abstraksi:

- Kelas Projectile mendefinisikan kerangka umum untuk peluru, namun tidak mengimplementasikan secara spesifik, memaksa kelas turunannya menyediakan implementasi.

I. ENKAPSULASI

Enkapsulasi merupakan konsep penting dalam pemrograman berorientasi objek yang berfungsi untuk melindungi data dalam suatu kelas agar tidak dapat diakses atau dimodifikasi secara langsung dari luar. Pada contoh kelas Projectile, prinsip ini diterapkan dengan menjadikan atribut `rect` dan `speed` sebagai bagian internal dari objek. Atribut `rect` merepresentasikan posisi serta ukuran proyektil dalam koordinat layar menggunakan objek dari pustaka `pygame`, sedangkan `speed` menentukan seberapa cepat proyektil bergerak. Untuk mengubah posisi proyektil, kelas ini menyediakan metode `update()` yang secara otomatis menambahkan nilai kecepatan ke koordinat horizontal (`x`) dari `rect`.

Dengan cara ini, setiap perubahan data dilakukan melalui metode internal, bukan secara langsung, sehingga lebih aman dan terkontrol. Pendekatan ini bertujuan untuk menjaga agar logika internal tetap tersembunyi dan mencegah kesalahan atau inkonsistensi akibat manipulasi langsung oleh bagian program lain.

J. PEWARISAN

Pewarisan merupakan salah satu prinsip utama dalam pemrograman berorientasi objek (OOP) yang memungkinkan sebuah kelas turunan (subclass) untuk mewarisi atribut dan metode dari kelas induk (superclass). Dalam contoh di atas, kelas `Fireball`, `Ice`, dan `Lightning` merupakan turunan dari kelas abstrak `Projectile`. Dengan mewarisi dari `Projectile`, ketiga kelas tersebut secara otomatis mendapatkan seluruh atribut seperti `rect` dan `speed`, serta dapat menggunakan atau memodifikasi metode yang ada pada kelas induknya.

Meskipun begitu, masing-masing kelas turunan mengimplementasikan metode `draw()` sesuai dengan jenis proyektilnya masing-masing, yaitu dengan menampilkan gambar proyektil tertentu pada posisi yang sesuai. Pendekatan ini sangat bermanfaat dalam menghindari penulisan kode yang berulang (redundansi), karena logika dasar pergerakan proyektil tidak perlu ditulis ulang di

setiap kelas turunan. Selain itu, pewarisan juga mempermudah proses pengembangan dan pemeliharaan kode, karena struktur program menjadi lebih terorganisir dan logis. Dengan menggunakan pewarisan, pengembang dapat menambahkan jenis proyektil baru dengan lebih efisien hanya dengan membuat kelas turunan baru yang sesuai.

K. POLIMORFISME

Polimorfisme adalah prinsip dalam pemrograman berorientasi objek yang memungkinkan suatu metode yang sama dipanggil pada objek-objek berbeda, namun menghasilkan perilaku yang berbeda sesuai dengan jenis objek tersebut. Dalam contoh kode, perulangan `for proj in projectiles:` digunakan untuk mengakses setiap objek dalam daftar proyektil, lalu memanggil metode `draw(win)` tanpa perlu mengetahui tipe objeknya secara eksplisit. Meskipun objek-objek tersebut bisa saja merupakan `Fireball`, `Ice`, atau `Lightning`, pemanggilan `proj.draw()` secara otomatis akan menjalankan implementasi metode `draw()` yang sesuai dengan tipe objek tersebut. Hal ini menunjukkan bagaimana satu nama fungsi (`draw`) dapat digunakan untuk berbagai objek yang memiliki perilaku berbeda. Keuntungan utama dari polimorfisme adalah fleksibilitas dalam menangani berbagai tipe objek dengan cara yang seragam, serta penyederhanaan kode karena tidak memerlukan logika bercabang seperti `if-else` untuk memeriksa tipe objek. Dengan kata lain, program menjadi lebih dinamis, mudah dikembangkan, dan efisien secara struktur.

L. ABSTRAKSI

Abstraksi adalah konsep penting dalam pemrograman berorientasi objek yang digunakan untuk menyembunyikan detail implementasi dan hanya menampilkan antarmuka atau struktur inti yang diperlukan. Dalam contoh ini, kelas `Projectile` dideklarasikan sebagai kelas abstrak dengan menggunakan modul `abc` dan decorator `@abstractmethod`. Hal ini berarti kelas `Projectile` tidak dapat dibuat objeknya secara langsung, tetapi hanya dapat diturunkan oleh kelas-kelas lain seperti `Fireball`, `Ice`, atau `Lightning`. Kelas abstrak ini menetapkan kerangka dasar berupa metode `draw()` yang wajib diimplementasikan oleh setiap subclass. Tujuan dari pendekatan ini adalah untuk memastikan bahwa semua jenis proyektil memiliki perilaku inti yang sama, yaitu kemampuan untuk menggambar dirinya di layar, namun masing-masing bebas menentukan cara implementasinya sendiri. Dengan menggunakan abstraksi, kompleksitas program dapat dikurangi karena struktur dasar sudah ditentukan dengan jelas, sekaligus meningkatkan konsistensi dan keteraturan dalam hierarki kelas proyektil.

M. PENANGANAN KESALAHAN

Dengan memeriksa setiap event yang ditangkap oleh `pygame.event.get()` dan mendeteksi apakah tipenya adalah `pygame.QUIT`, game dapat memberikan respon yang sesuai — dalam hal ini, mengembalikan nilai "exit" sebagai sinyal untuk menghentikan permainan. Pendekatan ini merupakan bagian penting dari sistem manajemen event yang aman, karena mampu mengantisipasi aksi pengguna seperti menutup jendela permainan atau menekan tombol tertentu.

Dengan demikian, game dapat menghindari terjadinya crash atau perilaku yang tidak diinginkan yang biasanya timbul akibat event yang tidak tertangani. Penanganan event seperti ini sangat penting untuk menjaga stabilitas aplikasi, memastikan bahwa interaksi pengguna tidak menyebabkan penghentian permainan secara tiba-tiba, serta memberikan pengalaman bermain yang lebih andal dan konsisten.

Catatan: Game ini belum menerapkan try-except secara eksplisit, namun sudah mengatur berbagai **kondisi logika** untuk mencegah error (seperti batas layar, cooldown, input).

N. GRAPHICAL USER INTERFACE

GUI yang Digunakan: Pygame

Game Battle of Heroes menggunakan Pygame sebagai library utama untuk membuat antarmuka grafis berbasis 2D. Pygame memungkinkan pengelolaan elemen-elemen visual dan interaksi pengguna secara langsung dalam lingkungan Python.

Dalam game ini, Pygame digunakan untuk:

- Menampilkan gambar karakter, musuh, latar belakang, dan proyektil melalui fungsi `blit()`
- Menggambar elemen UI seperti bar HP, skor, level menggunakan teks dan rectangle
- Mendeteksi input pengguna dari keyboard untuk navigasi menu dan kontrol karakter (tombol arah, Z, X, C, ESC)
- Menangani loop utama game, pengaturan frame rate, dan pembaruan layar (`pygame.display.update()`)
- Menampilkan menu interaktif, seperti menu utama, info, dan tampilan game over

GUI ini bersifat statik dengan animasi manual, artinya semua gerakan dan efek dibuat dengan mengatur ulang posisi objek secara berkala, bukan berbasis sistem GUI yang kompleks seperti tombol atau slider.

O. DIAGRAM UNIFIED MODELING LANGUAGE

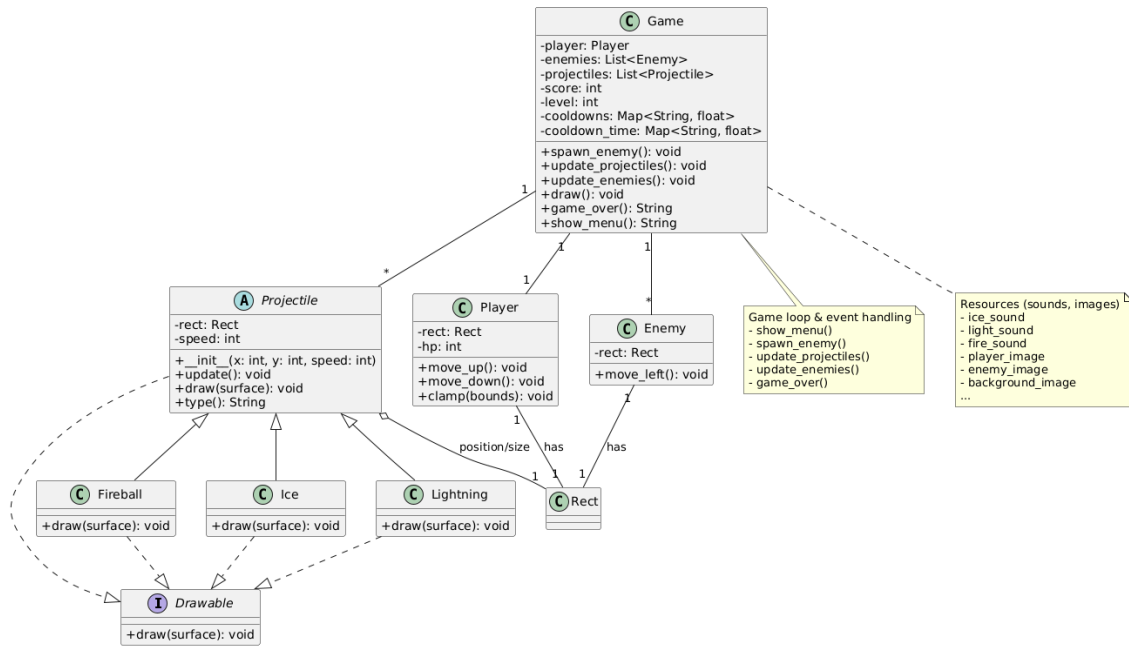
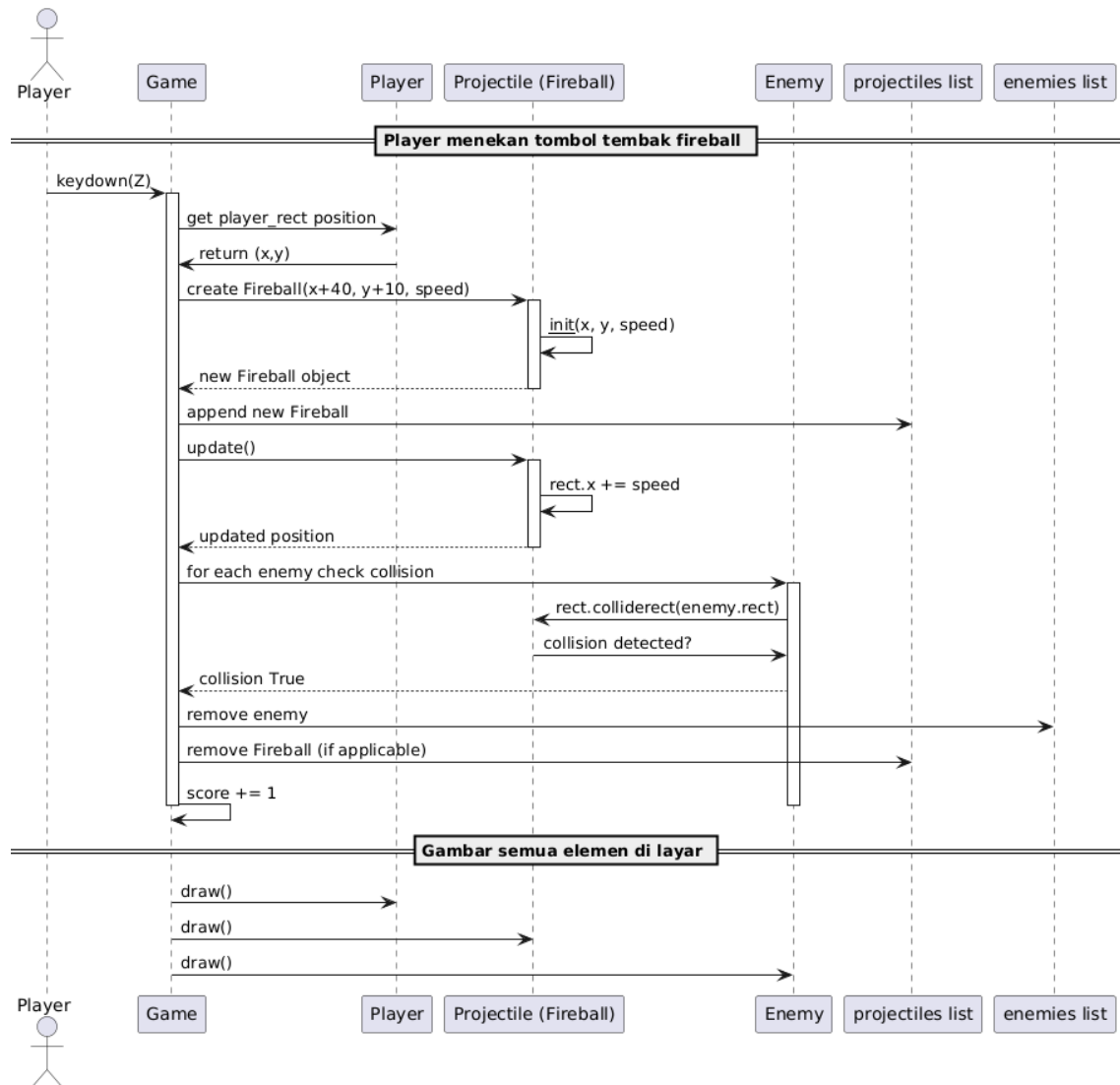
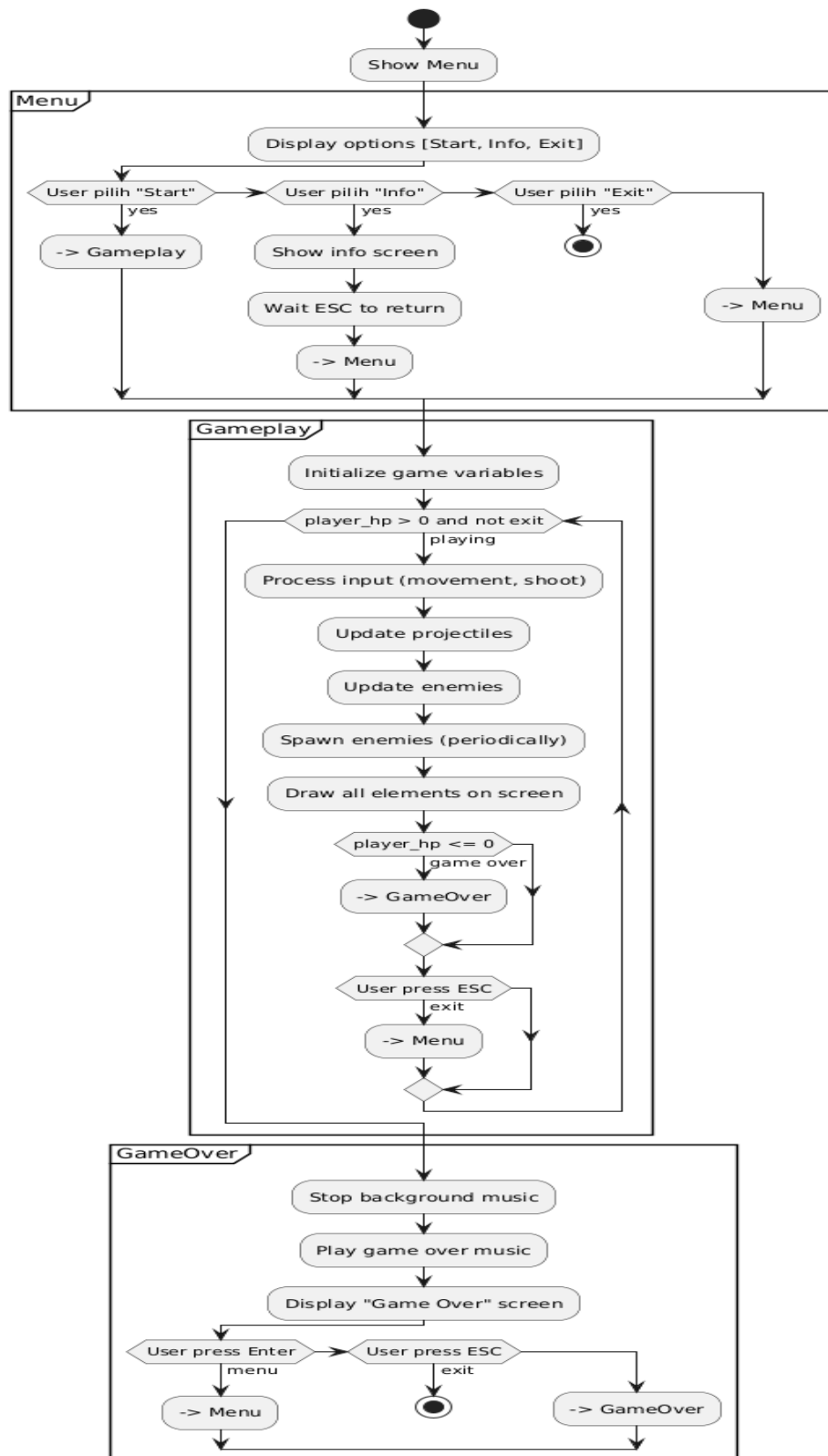


Diagram Sequence



Activity Diagram



P. KODE

1. Kelas Projectile dan Turunannya

- Abstraksi dan Pewarisan:

Kode ini menggunakan konsep abstraction dan inheritance di PBO. Kelas Projectile adalah kelas abstrak yang mendefinisikan struktur umum untuk proyeksi (seperti bola api, es, petir). Metode update() dan draw() di kelas ini diharapkan diimplementasikan oleh kelas turunan.

```
class Projectile(ABC):  
  
    def __init__(self, x, y, speed):  
  
        self.rect = pygame.Rect(x, y, 32, 32)  
  
        self.speed = speed  
  
  
    def update(self):  
  
        self.rect.x += self.speed  
  
  
    @abstractmethod  
    def draw(self, surface):  
  
        pass  
  
  
    def type(self):  
  
        return self.__class__.__name__
```

- Kelas Turunan Fireball, Ice, dan Lightning:

Kelas-kelas ini mewarisi Projectile dan mengimplementasikan metode draw dengan cara yang berbeda, sesuai jenis proyeksi yang mereka wakili. Ini merupakan contoh dari polymorphism.

```
class Fireball(Projectile):
```

```

def draw(self, surface):
    surface.blit(fireball_image, self.rect.topleft)

class Ice(Projectile):
    def draw(self, surface):
        surface.blit(ice_image, self.rect.topleft)

class Lightning(Projectile):
    def draw(self, surface):
        surface.blit(goblin_ice_image, self.rect.topleft)

```

2. Menggambar Elemen pada GUI

- Fungsi draw():
Fungsi ini bertugas menggambar semua elemen pada layar, seperti latar belakang, pemain, proyektil, musuh, dan statistik permainan seperti HP, skor, dan level. Fungsi ini memberikan pembaruan visual yang terintegrasi dengan cara yang efisien, memastikan semua elemen tampil dengan benar.

```

def draw():
    win.blit(background_image, (0, 0))
    win.blit(player_image, player_rect)

    for proj in projectiles:
        proj.draw(win) # Polymorphism: calls correct draw()

    for e in enemies:
        win.blit(enemy_image, e.topleft)

```

```

pygame.draw.rect(win, RED, (10, 10, player_hp * 2, 20))

win.blit(font.render(f'HP: {player_hp}', True, WHITE), (10, 35))

win.blit(font.render(f'Score: {score}', True, WHITE), (WIDTH - 150, 10))

win.blit(font.render(f'Level: {level}', True, WHITE), (WIDTH - 150, 35))


pygame.display.update()

```

3. Pengelolaan Musuh dan Proyektil

- Fungsi `update_projectiles()` dan `update_enemies()`:
Fungsi `update_projectiles()` memperbarui posisi proyektil dan memeriksa apakah proyektil mengenai musuh. Jika ya, musuh dihapus dan skor bertambah. Fungsi `update_enemies()` mengatur pergerakan musuh, serta memeriksa apakah musuh bertabrakan dengan pemain, yang akan mengurangi HP pemain.

```

def update_projectiles():
    global enemies, score

    for proj in projectiles[:]:
        proj.update() # Encapsulation: uses internal state

        for enemy in enemies[:]:
            if proj.rect.colliderect(enemy):
                enemies.remove(enemy)

                score += 1

                if proj.type() not in ("Ice", "Lightning"):
                    projectiles.remove(proj)

                break

        if proj.rect.x > WIDTH:

```

```

projectiles.remove(proj)

def update_enemies():
    global player_hp
    for e in enemies[:]:
        e.x -= 2
        if e.colliderect(player_rect):
            enemies.remove(e)
            player_hp -= 10
    elif e.x < 0:
        enemies.remove(e)
        player_hp -= 5

```

4. Menu Utama dan Fungsi show_menu()

- Navigasi Menu:
Fungsi show_menu() menangani tampilan menu utama dan pengaturan pilihan menggunakan tombol arah dan tombol enter. Ini juga menggunakan logika untuk menampilkan informasi tentang pembuat dan kontrol permainan jika pengguna memilih "Info".

```

def show_menu():
    selected = 0
    options = ["Start", "Info", "Exit"]
    showing_info = False

    while True:

```

```

win.blit(menu_background, (0, 0))

if showing_info:

    info_lines = [

        "Created by: Kelompok 2 PBO RA",

        "1. Doni Agus Setiawan",

        "2. Aprililianti",

        "3. Nabila Ramadhani Mujahidin",

        "4. Yuni Okta Safitri",

        "Controls:",

        " ↑/↓ = Move",

        " Z = Fireball",

        " X = Ice",

        " C = Lightning",

        " ESC = Exit",

        "Press ESC to go back."

    ]

    for i, line in enumerate(info_lines):

        win.blit(font.render(line, True, WHITE), (200, 250 + i*30))

else:

    for i, option in enumerate(options):

        color = YELLOW if i == selected else WHITE

        win.blit(font.render(option, True, color), (WIDTH//2 - 50, 250 + i*40))

pygame.display.update()

for event in pygame.event.get():

```

```

if event.type == pygame.QUIT:
    return "exit"

if event.type == pygame.KEYDOWN:
    if showing_info and event.key == pygame.K_ESCAPE:
        showing_info = False
    elif not showing_info:
        if event.key == pygame.K_UP:
            selected = (selected - 1) % len(options)
        elif event.key == pygame.K_DOWN:
            selected = (selected + 1) % len(options)
        elif event.key == pygame.K_RETURN:
            if options[selected] == "Start": return "start"
            if options[selected] == "Info": showing_info = True
            if options[selected] == "Exit": return "exit"

```

5. Fungsi `game_over()`

- Logika Game Over:

Fungsi ini menangani tampilan layar game over, menampilkan skor akhir, dan memberikan pilihan kepada pemain untuk kembali ke menu atau keluar dari permainan.

```

def game_over():
    while True:
        win.fill(BLACK)

```



```

msg = font.render("Game Over", True, WHITE)

final_score = font.render(f"Final Score: {score}", True, WHITE)

prompt = font.render("Press Enter to return to Menu or ESC to Quit", True,
WHITE)

win.blit(msg, (WIDTH//2 - msg.get_width()//2, 200))

win.blit(final_score, (WIDTH//2 - final_score.get_width()//2, 250))

win.blit(prompt, (WIDTH//2 - prompt.get_width()//2, 300))

pygame.display.update()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        return "exit"

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            return "menu"

        elif event.key == pygame.K_ESCAPE:
            return "exit"

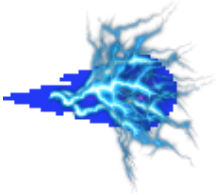
```

Kesimpulan:

Kode ini mencakup beberapa aspek penting dari pemrograman berorientasi objek, seperti pewarisan (inheritance), polimorfisme (polymorphism), dan enkapsulasi (encapsulation). Selain itu, penggunaan pygame untuk elemen GUI seperti menggambar gambar, menangani input pengguna, dan menampilkan teks juga menunjukkan bagaimana elemen visual diatur dalam game ini.

Q. LAMPIRAN

Karakter	Keterangan
	Nama Karakter: Tidak disebutkan eksplisit (dapat diasumsikan sebagai "Hero") Peran: Karakter utama yang dikendalikan oleh pemain Penampilan: Seorang penyihir dengan jubah, ditampilkan menggunakan sprite main.png Kemampuan: • Fireball 🔥 : Serangan api cepat dengan cooldown pendek • Ice ❄️ : Serangan pembekuan dengan cooldown menengah • Lightning ⚡ : Serangan petir yang kuat dengan cooldown terlama • Bergerak secara vertikal (atas-bawah) untuk menghindari musuh • Dapat meningkatkan level yang berdampak pada kecepatan serangan dan cooldown Ciri Khas: • Menyerang dengan kekuatan elemen sihir • Tugasnya adalah bertahan hidup selama mungkin dan mengalahkan musuh • Mewakili karakter pintar, strategis, dan adaptif terhadap situasi

	Goblin (Musuh) Nama Karakter: Tidak disebutkan eksplisit (disebut sebagai "enemy") Peran: Musuh yang menyerang pemain Penampilan: Goblin dengan tampilan khas monster, ditampilkan menggunakan sprite goblin.png Perilaku: • Muncul dari sisi kanan layar dan bergerak ke kiri • Bila bertabrakan dengan pemain, mengurangi HP pemain • Bila mencapai sisi kiri layar tanpa dihancurkan, juga mengurangi HP pemain Ciri Khas: • Tidak memiliki kemampuan menyerang langsung, hanya menyeruduk • Memiliki HP yang lemah namun muncul terus-menerus (musuh tipe spam) • Mewakili ancaman terus-menerus yang menguji kecepatan dan refleks pemain.
	Lightning • Cooldown: 5000 ms (5 detik) • Keterangan: Serangan sihir paling kuat, namun memiliki cooldown paling lama. Cocok digunakan untuk membasmi musuh dalam jumlah banyak atau saat dalam situasi genting.
	Fireball • Cooldown: 0 ms (tidak ada cooldown) • Keterangan: Serangan dasar yang cepat dan bisa ditembakkan terus-menerus. Efektif untuk menyerang musuh secara bertubi-tubi. Bisa dianggap sebagai senjata utama.

	Ice • Cooldown: 3000 ms (3 detik) • Keterangan: Serangan sihir dengan efek visual es, cooldown sedang, digunakan untuk mengontrol tempo pertempuran. Biasanya digunakan untuk menunda gerak musuh atau sebagai cadangan saat fireball kurang efektif.
---	---

Lampiran

Link youtube : <https://youtu.be/p92zN2evfWE?si=czs0UxiD5e0dwx4Y>